SECURITY

PKI & Encryption Visual Guide

Public keys, private keys, and the math that keeps secrets secret.

ANIMATED WALKTHROUGH

Step through pages to play the animation

STEP 1/6

Symmetric vs Asymmetric

Symmetric

AES-256

- One shared secret key
- Very fast (GB/s)
- Key distribution problem
- Use: bulk data encryption

Asymmetric

RSA / ECC

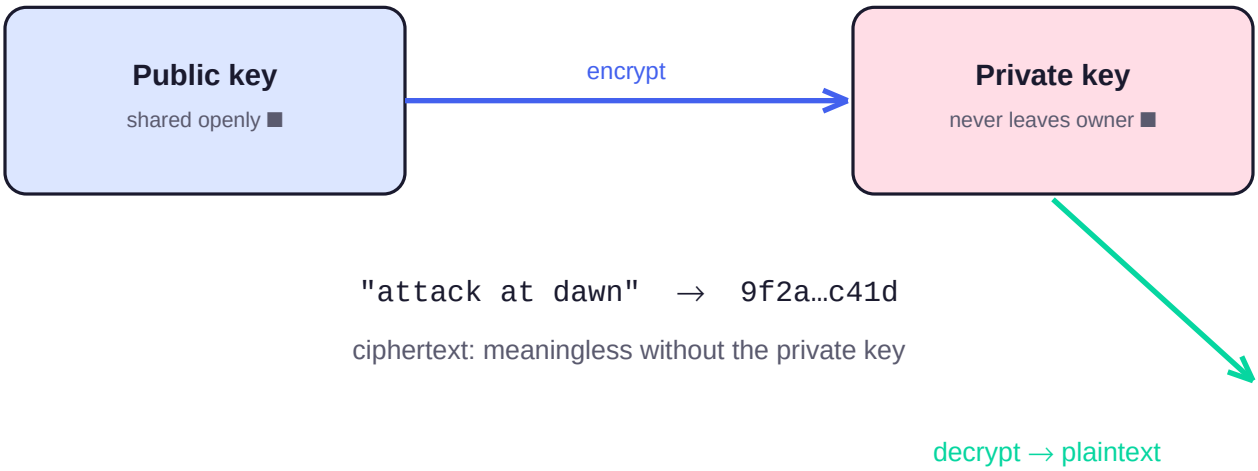
- Public + private key pair
- Slow (~1000x slower)
- Share public freely
- Use: key exchange, signatures

Hybrid reality: use asymmetric crypto to agree on a symmetric key, then encrypt bulk data symmetrically. Best of both.

STEP 2/6

Public-Key Encryption: The Mailbox Analogy

Anyone can drop a letter in; only the owner opens it.



STEP 3/6

Digital Signatures: Prove It Was You

Encryption's mirror image — authenticity instead of secrecy.

- 1. Hash the message (SHA-256) → fixed-size digest.
- 2. Encrypt the digest with the sender's PRIVATE key → signature.
- 3. Receiver decrypts the signature with the sender's PUBLIC key.
- 4. Receiver hashes the message itself and compares — match = authentic + untampered.
- Non-repudiation: only the private-key holder could have made that signature.

Sign with private → verify with public

the exact reverse of encryption

STEP 4/6

Certificates Bind Keys to Identities

A public key alone says nothing about WHO owns it.

X.509 Certificate

subject (example.com) • public key • issuer • validity • signature

A Certificate Authority (CA) verifies identity, then SIGNS the certificate.

Your browser trusts ~150 root CAs — their signatures vouch for everyone else.

STEP 5/6

Hashing: Fingerprints for Data

One-way functions — easy to compute, impossible to reverse.

```
ef92b778bafe771e89245b89ecbc08a44a4e166
c06659911881f383d4473e94f
```

- Deterministic: same input → same hash, always.
- Avalanche: one bit changes → ~half the output bits flip.
- One-way: can't reconstruct input from the hash.
- Passwords: store salted hashes (bcrypt/argon2), never plaintext, never plain SHA-256.

STEP 6/6

Where Each Primitive Lives

The crypto stack in one view.

TLS

ECDHE key exchange → AES-GCM bulk encryption → certs for auth

Passwords

Argon2/bcrypt salted hashes; never reversible encryption

Code signing

Developer signs with private key; OS verifies with public

End-to-end chat

Double-ratchet (Signal): forward secrecy per message

STEP ✓I✓

Key Takeaways

PKI & Encryption — Remember

- Symmetric (AES) is fast for bulk data; asymmetric (RSA/ECC) solves key distribution.
- Encrypt with PUBLIC, decrypt with PRIVATE. Sign with PRIVATE, verify with PUBLIC.
- Certificates bind a public key to an identity, vouched for by a CA chain.
- Hashes are one-way fingerprints — salt and stretch them for passwords (Argon2).
- Real systems are hybrid: asymmetric handshake, symmetric bulk, hashes for integrity.

Keep exploring

Pair this guide with the matching video walkthroughs
and hands-on labs at interstitiumlabs.dev